

***Universitatea Ovidius din Constanta
Facultatea de Matematica si Informatica
Specializarea Informatica***

Ingineria sistemelor software

Curs 12

Conf. univ. dr. Crenguta M. Puchianu
2020



Modelul de implementare

Acest model se obtine din translatarea arhitecturii software la nivel obiectual intr-un program orientat spre obiecte.

Este necesara scrierea codului sursa pentru:

- definirea claselor, obiectelor lor si a modului in care comunica obiectele intre ele;
- definirea interfetelor.

Tot in aceasta faza sunt rezolvate problemele aparute la scrierea codului sursa, compilarea si executarea lui.



Clasa ControllerCreatePermis

```
public class ControllerCreatePermis {
    private Permis p;
    private ManagerPermise mp;
    private FormularPermis fm;
    private Fereastrapermis f;
    public void cereCreatePermis(){
        p=null;
        fm=new FormularPermis(this);
        fm.afiseaza();
    }
    public void trimiteDate(String cnp,String nr,int perioada,String nume, String email){
        if(verificaSintactic(email)){

            if(!ManagerConturi.verificaCNP(cnp)){
                Cont c=new Cont(cnp, "cititor", email, cnp);
                ManagerConturi.memoreaza(c);
            }
            p=new Permis(cnp,nume,perioada,email,nr);
            f=new Fereastrapermis(this);
            f.afiseaza(p.toString());
        }
    }
}
```



```

private boolean verificaSintactic(String e){ return e.contains("@"); }
public boolean verificaCnp(String cnp){
    if(cnp.length()!=13)return false;
    return true;
}
public void confirma(String s){
    PrinterJob imprimanta=PrinterJob.getPrinterJob();
    Book bk=new Book();
    bk.append(new PaintContent(s),imprimanta.defaultPage());
    imprimanta.setPageable(bk);
    if(imprimanta.printDialog()){
        try{imprimanta.print();}catch (PrinterException
        pe){JOptionPane.showMessageDialog(null,"Imprimanta nu exista");}
        catch(ArrayIndexOutOfBoundsException ae){System.out.println("Ce se
        prindeaza???");}
    }
    ManagerPermise mp=ManagerPermise.getInstance();
    if(p!=null) if(mp.memoreaza(p)){
        JOptionPane.showMessageDialog(null,"permisul a fost memorat");
        f.dispose();
        fm.dispose();
    }
    else JOptionPane.showMessageDialog(null,"Eroare la memorarea permisului");
}

```



```

class PaintContent extends JPanel implements Printable
{ private BufferedReader br;
    private String sir="",s;
public PaintContent(String s){ this.s=s; }
public int print(Graphics g, PageFormat pf,int pageIndex) throws PrinterException{
    g.setColor(Color.black);
    try{
        StringReader continut=new StringReader(s);
        br=new BufferedReader(continut);
        int i=0;
        while((sir=br.readLine())!=null) {
            if (sir.length()==0) sir=" ";
            g.drawString(sir,100,100+i);i+= 20;
        }
    }catch(IOException io){}
    catch (IllegalArgumentException ie){}
    return Printable.PAGE_EXISTS;
}

public void anuntaAnulare(){
    JOptionPane.showMessageDialog(null,"Permisul nu a fost memorat in baza de date");
    f.dispose();
}
}

```



Clasa Permis

```
package CrearePermis;
import CreareCont.*;
public class Permis {
    private Cititor c;
    private int numar,valabilitate;
    private static int id=ManagerPermise.getInstance().getIdPermis();
    public Permis(String cnp,String nume,int perioada,String email, String nrTelefon){
        numar=++id;
        String[] itemi=nume.split(" ");
        valabilitate=perioada;
        c=new Cititor(cnp, itemi[0],itemi[1],email, nrTelefon);
    }
    public String toString(){
        return numar+" "+c.getCNP()+" "+c.getNum()+" "+c.getEmail()+"
"+valabilitate+" "+c.getNrTel();
    }
    public int getNumarPermis(){
        return numar;
    }
}
```



Clasa Cititor

```
public class Cititor {
    private Persoana p;
    private String cnp,nrTel;
    public Cititor (String cnp,String nume,String prenume){
        this.cnp=cnp;
        p=new Persoana(nume,prenume, nrTel, cnp);
    }
    public Cititor (String cnp,String nume,String prenume,String email,String nrTel){
        this.cnp=cnp;
        p=new Persoana(nume,prenume, email, cnp);
        this.nrTel=nrTel;
    }
    public String getCNP(){
        return cnp;
    }
    public String getNume(){
        return p.getNume();
    }
    public String getNrTel(){
        return nrTel;
    }
    public String getEmail(){return p.getEmail();}
}
```



Clasa ManagerPermise

```
public class ManagerPermise {  
    private static ManagerPermise mp;  
    private static StringBuffer data;  
    private static int idPersoana, idPermis;  
  
    public boolean memoreaza(Permis p){  
        Conexiune c=Conexiune.getInstanta();  
        Statement stmt=c.createStatement();  
        String s=p.toString();  
        String[] itemi=s.split(" ");  
  
        try{  
            PreparedStatement ps=c.getConexiune().prepareStatement("insert into  
persoane(cnp,nume,prenume,email) values(?,?,?,?);",  
Statement.RETURN_GENERATED_KEYS);  
            ps.setString(1, itemi[1]);  
            ps.setString(2, itemi[2]);  
            ps.setString(3, itemi[3]);  
            ps.setString(4, itemi[4]);  
            ps.executeUpdate();  
            ResultSet cheie=ps.getGeneratedKeys();  
            if (cheie.next()) idPersoana=cheie.getInt(1);  
        } catch(SQLException e){e.printStackTrace(); return false;}
```



Clasa ManagerPermise (cont.)

```
String comanda="insert into cititori(idPersoana,nrTelefon)
values("+idPersoana+","+itemi[6]+"");
    Calendar gc=Calendar.getInstance();
    data=new StringBuffer();
    data.append(gc.get(Calendar.DAY_OF_MONTH));
    data.append("/") +gc.get(Calendar.MONTH));
    data.append("/") +gc.get(Calendar.YEAR));
    try{
        stmt.executeUpdate(comanda);

        PreparedStatement ps=c.getConexiune().prepareStatement("insert into
permise(valabilitate,idCititor,data) values(?,?,?);");
        ps.setString(1, itemi[5]);
        ps.setInt(2, idPersoana);
        ps.setString(3, data.toString());
        ps.executeUpdate();
        return true;

    }catch(SQLException e){e.printStackTrace();}
    return false;
}
```



Clasa ManagerPermise (cont.)

```
public static ManagerPermise getInstance(){
    if(mp==null)mp=new ManagerPermise();
    return mp;
}

public int getIdPermis(){
    Conexiune c=Conexiune.getInstanta();
    Statement stmt=c.createStatement();
    String comanda="select MAX(id) from permise";
    try{
        ResultSet rs=stmt.executeQuery(comanda);
        if(rs.next()) idPermis=rs.getInt(1);
    }catch(SQLException e){e.printStackTrace();}
    return idPermis;
}
}
```



Clasa FormularPermis

```
public class FormularPermis extends JFrame{
    private JButton b1,b2;
    private JTextField tf1,tf2,tf3,tf4, tf5;
    private JPanel p2,p3;
    private ControllerCrearePermis cc;
    public FormularPermis(ControllerCrearePermis cc){
        setTitle("formular permis");
        this.cc=cc;
        tf1=new JTextField();   tf2=new JTextField();   tf3=new JTextField();   tf4=new
JTextField();   tf5=new JTextField();
        b1=new JButton("ok");   b2=new JButton("cancel");
        p2=new JPanel();   p2.setLayout(new GridLayout(5,2,10,10));
        p2.add(new JLabel("cnp"));   p2.add(tf1);
        p2.add(new JLabel("nume"));   p2.add(tf2);
        p2.add(new JLabel("telefon"));   p2.add(tf3);
        p2.add(new JLabel("email"));   p2.add(tf5);
        p2.add(new JLabel("perioada valabilitate"));   p2.add(tf4);
        p3=new JPanel();   p3.add(b1);   p3.add(b2);
        AscultatorButoane ab=new AscultatorButoane();
        b1.addActionListener(ab);
        b2.addActionListener(ab);
        this.add(p2);
        this.add(p3,BorderLayout.SOUTH);
    }
}
```



Clasa FormularPermis (cont.)

```
private class AscultatorButoane implements ActionListener{
    public void actionPerformed(ActionEvent e){
        if(e.getSource()==b1){
            cc.trimiteDate(tf1.getText(), tf3.getText(), Integer.parseInt(tf4.getText()),
                tf2.getText(), tf5.getText());
        }else{FormularPermis.this.dispose();}
    }
}

public void afiseaza(){
    this.setSize(400,400);
    this.setVisible(true);
    this.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
}
}
```



Clasa FereastraPermis

```
class FereastraPermis extends JFrame{
    private JLabel l;
    private JTextArea ta;
    private JButton b1,b2;
    private ControllerCrearePermis ccp;
    public FereastraPermis(ControllerCrearePermis ccp){
        this.ccp=ccp;
        setTitle("Date permis");
        l=new JLabel("Permis inregistrat");
        ta=new JTextArea(20,50);
        JPanel p1=new JPanel();
        b1=new JButton("printeaza");
        b2=new JButton("cancel");
        AscultatorButoane ab=new AscultatorButoane();
        b1.addActionListener(ab);
        b2.addActionListener(ab);
        p1.add(b1);
        p1.add(b2);
        this.add(l,BorderLayout.NORTH);
        this.add(ta);
        this.add(p1,BorderLayout.SOUTH);
    }
}
```



Clasa FereastraPermis (cont.)

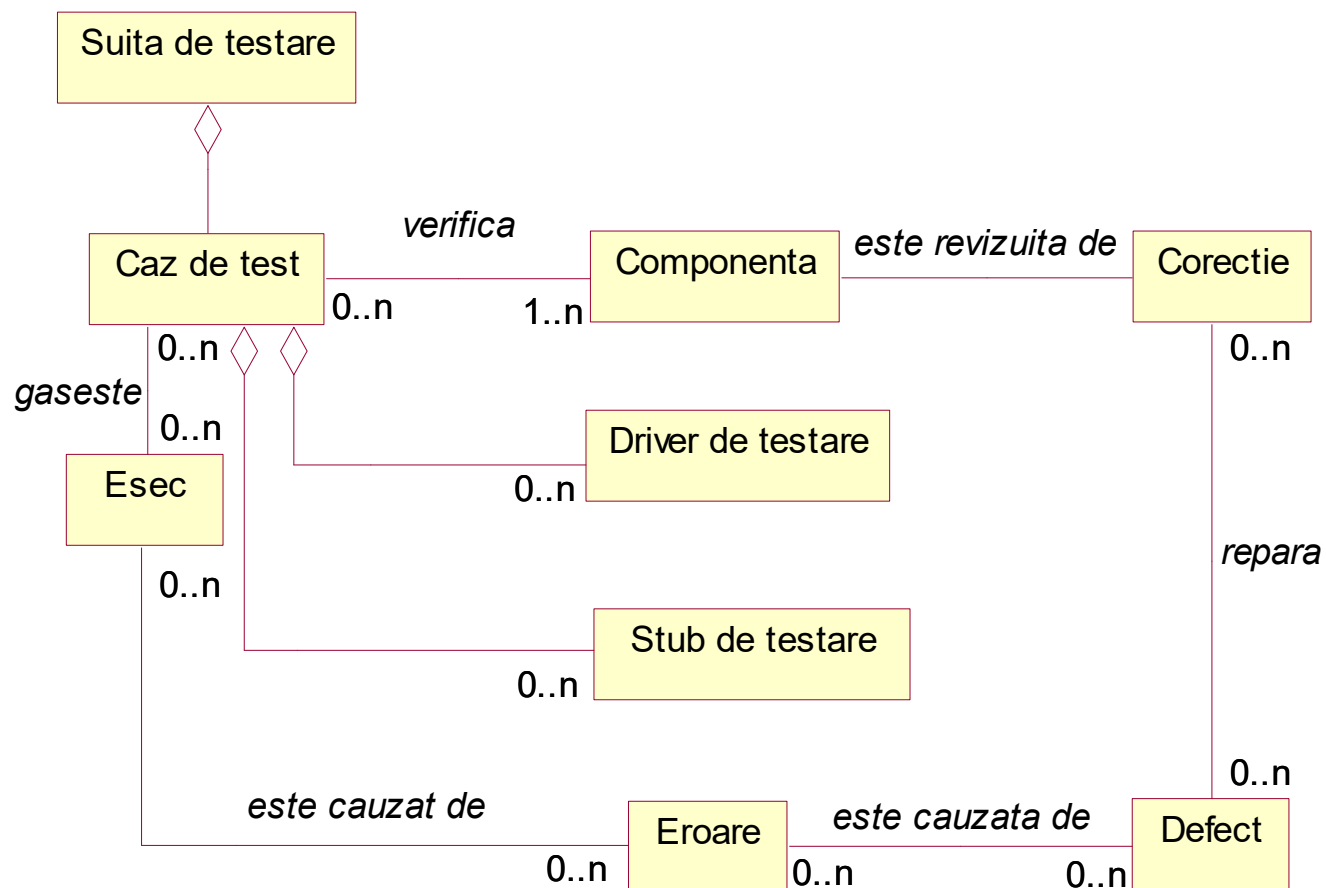
```
private class AscultatorButoane implements ActionListener{
    public void actionPerformed(ActionEvent e){
        if(e.getSource()==b1){
            ccp.confirma(ta.getText());
        }
        else{
            ccp.anuntaAnulare();
            FereastraPermis.this.dispose();
        }
    }
}

public void afiseaza(String datePermis){
    ta.setText(datePermis);
    this.setSize(300,300);
    this.setVisible(true);
    this.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
}
```



Testarea sistemelor software

Testarea este procesul de gasire a diferentelor intre comportamentul asteptat al sistemului asa cum a fost descris de modele si comportamenul observat al sau.



Testarea bazata pe unitati

Sensul de testare din perspectivei cutiei albe este de „a testa pentru a implementa”: adica studierea codului, intelegrea lui si testarea cailor de executie posibile. Asadar, testarea bazata pe unitati gaseste diferentele dintre modelul obiectual obtinut in urma proiectarii si modelul implementarii.

O unitate de test are doua roluri: o tinta a testului sau o resursa a unui test.

O tinta a unui test este de obicei o clasa in codul aplicatiei, dar poate fi formata din cateva metode ale unei clase sau cateva clase care coopereaza.

Ca resursa, o unitate de testare este o bucata de cod – o clasa sau o multime de clase care realizeaza testarea. Metodele unei clase unitate de testare se numesc cazuri de test.

Cazurile de test pot fi combinate in suite de test pentru a rula o colectie de cazuri de test care au ca tinta mai multe clase sau chiar intregul sistem.

Testarea unitatilor furnizeaza urm. avantaje:

- reduce complexitatea testarii intregului sistem, permitandu-ne sa ne concentram asupra componentelor mai mici ale sistemului
- faciliteaza gasirea bug-urilor si corectarea lor
- permite realizarea in paralel a activitatilor de testare.



Testarea de acceptare

Un test de acceptare contine urmatoarele elemente:

- un script de test
- implementarea scripului de test (daca e posibil)
- specificarea datelor de intrare folosite pentru testare
- documentul rezultatului testului

Testele de acceptare sunt scrise ca scripturi (de testare). Scripturile pot fi automatizate complet sau partial. Orice esec al sistemul descoperit prin aceasta testare sunt impartite in 2 categorii:

- defecte (bug-uri) si sunt rezolvate in versiunea curenta
- imbunatatiri si vor fi considerate in iteratii viitoare.

Similar, testele de acceptare implementate sunt organizate in cazuri de test si suite de test cu urmatoarele modificari:

- de obicei, o unitate a unui test de acceptare implementeaza o parte de functionalitate ce cuprinde mai multe clase.
- Testele de acceptare contin puncte de verificare, in care verificam daca functionalitatea asteptata este realizata de sistemul software.



Scripturi de test

Un script de test este scris ca un tabel cu urmatoarele coloane: Linie, Verificare automata, Descriere. Tabelul arata pasii testului si punctele de verificare: Pi-pasul i, Vi- punctul de verificare i.

Linie	Verificare automata	Descriere
P1 V1	true	Solicita crearea unui permis A fost afisat formularul permisului?
P2 V1	true	Introduce date corecte Au fost afisate datele permisului?
P3 V1	true	Introduce date incorecte A fost afisat un mesaj de eroare si a permis utilizatorului reintroducerea datelor?
P4 V1	true	Cere printarea permisului Permis a fost listat la imprimanta si a fost afisat un mesaj ca datele permisului au fost memorate in baza de date?
P5 V1	true	Cere anulara furnizarii permisului A fost afisat un mesaj de atentionare si datele permisului nu au fost memorate in baza de date?



Scripturi de test (cont.)

Scripturile de test verifica o parte din functionalitatea sistemului.

De exp, scriptul anterior testeaza daca sistemul indeplineste cerintele functionale ale sistemului cand este executat cazul de utilizare Creare permis:

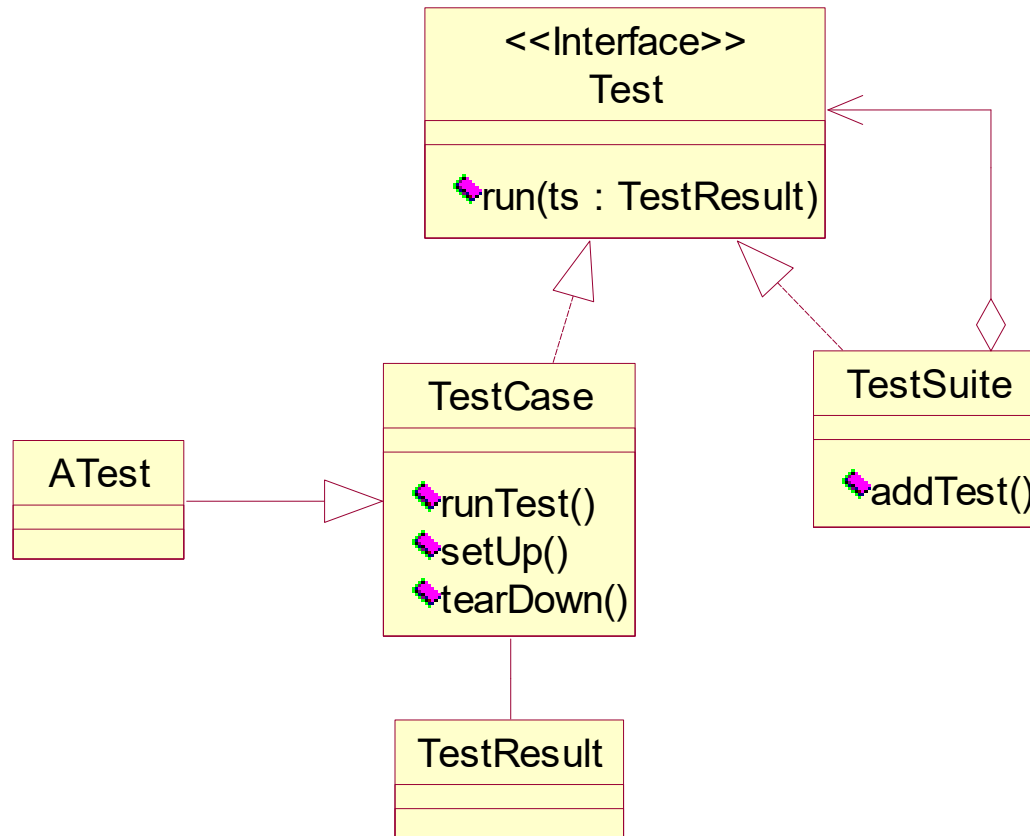
- A afisat un formular;
- A verificat sintactic si semantic datele cititorului;
- A creat contul cititorului in cazul in care este un cititor nou;
- A afisat datele permisului create;
- A tiparit permisul la imprimanta;
- A memorat datele permisului si ale cititorului asociat;
- A afisat un mesaj de terminare cu success.



JUnit

Este un framework pentru crearea cazurilor de testare, a suitelor de testare si a instrumentelor necesare executarii lor. Clasele principale sunt TestCase si TestSuite. Ambele realizeaza interfata Test.

Framework-ul se bazeaza pe modelul de proiectare Composite.



Clasa Assert a lui JUnit

Metoda	Efect
<code>void assertEquals(double valAsteptata, double valActuala)</code>	Afirma ca valoarea asteptata este egala cu valoarea actuala
<code>void assertEquals(double valAsteptata, double valActuala, double eps)</code>	
<code>void assertNotNull(Object o)</code>	afirma ca variabila o nu are valoarea null. Altfel se arunca exceptia AssertionError
<code>void assertSame(String m, Object obAsteptat, Object obActual)</code>	afirma ca parametri obAsteptat si obActual refera acelasi obiect. Altfel, se arunca exceptia AssertionError impreuna cu mesajul m transmis ca parametru al metodei
<code>void assertTrue(boolean c)</code>	afirma ca expresia logica c este adevarata. Altfel se arunca exceptia AssertionError

Adnotari:

@Test – marcheaza o metoda ce va fi testata

@Before – o metoda adnotata cu **@Before** se executa inaintea executarii fiecarei metode test.

@After – o metoda adnotata cu **@After** se executa dupa executarea fiecarei metode



Suite de test

O suitea de teste este o agregare formata din mai multe teste executate.

Varianta I:

```
import junit.framework.*;

public class TestSistem {
    public static Test suite(){
        TestSuite suitea = new TestSuite();
        suitea.addTestSuite(TestControllerCrearePermis.class);
        // sunt adaugate alte teste aici
        return suitea;
    }

    public static void main(String[] args){
        junit.textui.TestRunner.run(suite());
    }
}
```



Suite de test (cont.)

Varianta 2: Crearea unei suite de teste a claselor unui proiect NetBeans:

P1. Se alege pachetul pentru care generam suita de teste

P2. Click dreapta si alegem Tools -> Create/Update Tests

P3. In fereastra aparuta alegem locatia in care vor fi create clasele de testare. Implicit, este TestPackages si apasam butonul OK.

P4. In fereastra aparuta alegem versiunea de JUnit ce va fi folosita si OK

P5. Se modifica fiecare test.

P6. Se compileaza si se executa.



Manualul utilizatorului

Acest ghid descrie în detaliu pașii pe care trebuie să-i urmeze fiecare tip de utilizator pentru a utiliza corect sistemul dezvoltat.

Poate fi structurat după următoarele criterii:

- Actor software
- Grupuri de cazuri de utilizare



Validarea

Este activitatea finala care cuprinde, evident, integrarea sistemului, acceptarea din partea clientului si instalarea sistemului in mediul său natural. Validarea se concentreaza asupra verificarii aspectelor logice interne ale sistemului in vederea garantarii ca toate functiile sistemului au fost implementate in conformitate cu cerintele.

In acest punct, sistemul trebuie sa functioneze conform cerintelor utilizatorului, actualizate sau nu in timpul dezvoltarii.

În faza de validare sistemul este supus unor teste (de validare) care ar trebui sa confirme corectitudinea functionarii sistemului în raport cu cerintele utilizatorilor si plasarea performantelor acestuia în limitele stabilite de acele cerinte.

Programul împreuna cu documentatia aferenta este supus unui proces de verificare constituit din teste de diferite tipuri în vederea confirmarii corectitudinii din punct de vedere ale cerintelor initiale, eventual modificate pe parcursul dezvoltarii.

Verificarea se poate face în diverse medii de test, la furnizor sau la beneficiar, dar ultimul test trebuie facut în mediul natural în care va functiona produsul. Acest ultim test se numeste *validare (validation)* si trebuie condus de catre beneficiarul sistemului împreuna cu furnizorul lui. In vederea trecerii acestui test sistemul software trebuie integrat în mediul sau constituit din hardware, baze de date si alte sisteme cu care trebuie sa comunice.



Testarea utilizabilitatii

Identifica diferentele dintre sistem si asteptările utilizatorilor cu privire la comportamentul acestuia.

Tehnica de realizare a testelor privind utilizabilitatea este următoarea:

1. Dezvoltatorii formuleaza un set de obiective descriind informatia pe care se asteapta sa o obtina în urma testelor. De exemplu: evaluarea interfetei grafice, evaluarea masurii în care manualul utilizatorului sau documentatia raspunde nevoilor utilizatorilor)
2. Obiectivele anterioare sunt evaluate într-o serie de experimente în care utilizatorii sunt pregatiti sa execute anumite sarcini
3. Dezvoltatorii observa participantii si colecteaza date privind performantele acestora (timp de îndeplinire a unei task, rata erorilor) si preferintele lor

Tipuri de teste de utilizabilitate:

- Teste bazate pe scenarii (ale cazurilor de utilizare software)
- Teste bazate pe prototipuri
- Teste pe baza sistemului real

În urma testării utilizabilității sistemului de către utilizatori, aceștia completează un chestionar de utilizabilitate pregătit de către dezvoltatori.



Intretinerea sistemului software

Intretinerea se ocupa de modificarea produsului sau sistemului in cursul utilizarii lui.

Modificarile se datoreaza fie eliminarii unor erori de constructie, fie unor actualizari sau adaptari ale lui la noi conditii de lucru. In ultimul caz modificarile provin de la cel care a comandat realizarea produsului si vizeaza satisfacerea unor noi exigente aparute pe parcurs.

Este activitatea cea mai de lunga durata si mai costisitoare din evolutia oricarui sistem.

In aceasta faza sistemul se prezinta ca o entitate dinamica in continua modificare.

Modificarile se datoreaza fie ajustarii lui pentru a corespunde cerintelor initiale prin eliminarea bug-urilor, cat si adaptarii lui la noi cerinte ce provin din partea clientului.

